

Implementation and validation of SIMPLE for STM32 Microcontrollers

Filippo Aghi
University of Padua
2200442

Abstract—Remote attestation allows a trusted verifier to assess the software state of a remote embedded device. This project implements a practical SIMPLE attestation between two microcontrollers: an STM32H743VIT6 prover that measures its approved flash region and displays the result, and an STM32G431CBU6 verifier that issues fresh challenges and validates authenticated responses. The devices communicate through a deterministic UART protocol that provides freshness, replay protection, and failure handling. An offline provisioning step derives the expected measurement from the final prover firmware and produces matched deployment artifacts. The system is implemented and tested on real hardware using a segmented flashing procedure tailored to each device. While the protocol flow follows SIMPLE, the design omits the Security MicroVisor and therefore targets a narrower trusted firmware model.

I. OBJECTIVES

A. Motivation and project scope

Resource constrained IoT devices often lack the secure boot, trusted execution and hardware root of trust facilities available on general purpose systems. Remote attestation (RA) addresses part of this problem by allowing a trusted verifier to challenge a prover and evaluate whether the prover's measured software state matches an approved reference.

The reference paper introduces SIMPLE, an RA protocol designed for constrained devices and built on the Security MicroVisor ($S\mu V$) [1]. Its protocol uses two shared keys, a monotonically increasing counter, a fresh nonce, and a keyed measurement of device state.

The objective of this work was to produce a functional and reviewable implementation for two concrete roles:

- an STM32H743VIT6 prover, which performs the more expensive flash measurement and reports phases on its TFT display;
- an STM32G431CBU6 verifier, which creates fresh challenges, tracks one outstanding session, authenticates the response, and signals successful verification;

B. Development objectives and contributions

The work focused on implementing the SIMPLE protocol on real STM32 microcontrollers, validating its behavior under actual hardware constraints, and assessing its limits when executed without the $S\mu V$ isolation layer. The goal was to translate the abstract attestation logic into a concrete bare metal system and observe its practical properties.

The development was structured around three high level objectives:

- 1) Reconstruct and implement the protocol logic from the reference paper, mapping roles, messages, measurements, and counters onto STM32G4 and STM32H7 in a baremetal environment.
- 2) Ensure protocol correctness by enforcing proper authentication bindings, nonce usage, counter progression, flash measurement, and deterministic wire communication.
- 3) Provide a functional end-to-end system with verifier, prover, persistent storage, UART transport, build structure, and operational documentation suitable for evaluating SIMPLE in a non-isolated setting.

This structure enabled a complete and coherent implementation of the protocol on physical hardware.

C. Security objective and limits

The implementation aims to ensure four operational properties. First, an unauthenticated party must not be able to create a request accepted by the prover or a positive response accepted by the verifier. Second, a previously valid request or response must not be reusable in a later session. Third, the positive result must correspond to the configured H743 flash region and the verifier's approved reference. Fourth, reset or communication loss must not cause a previously issued counter to be reused.

These properties depend on the secrecy of K_{auth} and K_{attest} and also on the integrity of the trusted firmware, correct random number generation, monotonically increasing persistent state and correct STM32 protection configuration. The implementation uses HMAC-SHA-256 and SHA-256.

However, the security boundary is narrower than the one in the paper. SIMPLE derives its malware resistance and key isolation claims from $S\mu V$, including formally verified memory safety and controlled entry points. Here, all firmware executes as one privileged image. The MPU marks SRAM non executable and protects flash regions, while RDP and WRP option bytes limit readout and rewriting. These measures are useful for a trusted image deployment, but malicious privileged code could reconfigure the MPU or access credentials in RAM. This report therefore does not claim equivalence to $S\mu V$, resistance to physical attacks or fault injection and formal memory safety.

D. Protocol definition

II. PROTOCOL OVERVIEW

At provisioning time, both devices receive the shared authentication key K_{auth} , the attestation key K_{attest} , and the same initial counter. The verifier also stores the reference measurement

$$V_S = \text{HMAC-SHA256}_{K_{\text{attest}}}(M_{\text{approved}}),$$

where M_{approved} is the immutable firmware image over the fixed measurement interval.

A session begins when the verifier generates a fresh nonce N and the next monotonic counter C_V . It constructs a request containing the counter, the reference measurement, and the nonce, together with an authentication tag under K_{auth} . The verifier persists C_V before transmission and retains (C_V, N) as the outstanding challenge.

The prover accepts the request only if its current counter C_P is strictly lower than C_V then commits $C_P \leftarrow C_V$, measures its current firmware to obtain

$$V'_S = \text{HMAC-SHA256}_{K_{\text{attest}}}(M_{\text{current}}),$$

and returns a one-bit result indicating whether $V'_S = V_S$, authenticated together with the counter and nonce under K_{auth} . The verifier accepts the response only for its outstanding challenge, within the deadline, with a valid tag. Once accepted, the challenge is consumed; malformed traffic cannot advance counters or reset timing. Counter wrap is forbidden.

III. SYSTEM SETUP

A. Hardware roles and communication

Table I describes the configured instances. The H743 is assigned the prover role and it performs the flash HMAC. The G431 acts as verifier and initiates a challenge approximately two seconds after boot, then starts another session 60 seconds after the previous completion or timeout. Although the verifier drives the protocol, assigning this role to the G431 is intentional. The verifier's workload is deliberately lightweight: it authenticates frames, advances its counter, and issues challenges, making even a modest Cortex-M4 fully adequate. This reflects the design goal of SIMPLE, where the verifier can remain computationally small while supervising complex systems. The H743 performs the flash measurement only because it was chosen as prover, not because the task intrinsically requires an M7 class device; a G4 could sustain it as well, but the assignment remains symbolic in terms of role separation.

The peers use USART2 at 115200 baud with eight data bits, no parity and one stop bit.

The display shows the current prover phase: READY, REQUEST, MEASURING, PASS, which indicates that its comparison matched. Failures are reported with a specific error identifier while the G431 LED turns on only after an authenticated matching response.

TABLE I
CONFIGURED BUILD INSTANCES.

Instance	Purpose and configuration
stm32h743vit6	Cortex-M7 prover; 2 MiB flash; 128 KiB measurement; HSI48 backed RNG, TIM2, USART2 on PA2/PA3, and ST7735 TFT over SPI4.
stm32g431cbu6	Cortex-M4 verifier; 128 KiB single bank flash and 32 KiB RAM; 8 MHz HSE configured for a 170 MHz system clock; HSI48 backed RNG, TIM2, USART2 on PA2/PA3, and status LED on PC6.

B. Software architecture

The architecture is organized around a clear separation between portable protocol logic and hardware dependent code. The core module implements serialization, authentication, measurement, state progression, framing, and persistent freshness without any awareness of STM32 registers, memory maps, or peripherals. This separation is expressed directly in the structure of the implementation.

Prover application	Verifier application
Portable auth_core serialization HMAC/attestation state machines COBS transport authenticated counter journal	
Platform Abstraction Layer storage RNG clock atomic section UI byte stream	
Platform Mapping Layer H743VIT6 prover: USART2, TIM2, RNG, SPI4/TFT G431CBU6 verifier: USART2, TIM2, RNG, flash journal	

Fig. 1. Layered organization of the implementation.

The `auth_core` target contains protocol encoding, TinyCrypt wrappers, streaming memory measurement, prover and verifier state machines, COBS framing, and a two block counter journal. The application state machines receive an `auth_platform_t` containing callbacks for material loading, persistent counter commit, random bytes, time, atomic entry and exit, and UI notifications. An `auth_link_t` supplies a nonblocking byte read and a bounded buffer write.

The STM32 implementation provides these callbacks through HAL completing the low level logic. This keeps protocol execution predictable and prevents device specific behavior from leaking into the core. The same layering guides the build and deployment workflow, where each instance selects its device configuration and role while sharing the same portable logic. Provisioning and journal handling remain independent from the executable image and preserve the physical erase state of flash, producing a compact and consistent structure where hardware details remain confined to the lower layer.

C. Encoding and framing

The protocol uses a compact and fixed layout wire format. All fields have exact lengths and both endpoints validate size and structure before any semantic processing. Requests authenticate the verifier's counter, the approved measurement digest and a fresh nonce; while responses authenticate a one bit result bound to the prover's counter and the challenge.

UART payloads are wrapped as

$$\text{framing COBS}(\text{type}[1] \parallel \text{length}[1] \parallel \text{payload}) \parallel 0 \times 00). \quad (1)$$

While communicating a sender first emits an extra zero delimiter so that a peer can resynchronize. Then the decoder is able to reject invalid COBS runs, invalid type/length pairs, capacity overruns, and delayed frames. Using HMAC, no CRC is needed in this context.

D. Measurement and flash partitioning

The H743 measurement is the interval $[0 \times 08000000, 0 \times 08020000)$. It includes all linked firmware plus $0 \times \text{FF}$ padding up to 128 KiB. The linker rejects an image that exceeds this interval.

The provisioning phase produces a 160 byte record containing a magic value, version, role, two 32 byte keys, reference state, measurement start and length, initial counter, and a SHA-256 corruption checksum. Authenticity of the record relies on trusted physical provisioning and write protection. *Runtime code never rewrites credentials.*

The four flash partitions chosen have separate duties. The firmware partition contains executable code. The provisioning partition holds immutable role and key material plus the verifier's reference state. Journal A and Journal B alternate as append only counter stores. Keeping credentials and journals outside the executable interval allows for different write protection policies: firmware and provisioning are protected after installation, while journal pages remain writable.

Both blocks receive one seed record during provisioning. This information allows to evaluate memory at boot, and every non erased record is authenticated and checked for strict order, size, and gaps that could cause a forced failed initialization.

IV. EXPERIMENTS

A set of practical experiments was conducted to validate the protocol's behavior under normal operation, controlled faults, and forced desynchronization. After several successful attestations [Fig. 2], the prover was reflashed with a different firmware and provisioning, producing the expected NO_AUTH [Fig. 3]. Restoring the original firmware and provisioning caused the next attestation to PASS, revealing a behavior not explicitly discussed in the paper.



Fig. 2. Successful attestation (PASS).



Fig. 3. Authenticated mismatch (NO_AUTH).

Additional experiments included forced UART corruption [Fig. 4], truncated frames, power interruption during measurement, and deliberate desynchronization. In all cases the prover rejected malformed input and the verifier produced UART ERROR, and the protocol resynchronized cleanly once valid authenticated traffic resumed.



Fig. 4. UART corruption error.

V. RESULTS AND DISCUSSION

The implementation behaves as intended and reaches the practical goals defined for this SIMPLE style attestation. The prover produces authenticated results, then rejects stale or malformed input and maintains consistent state across sessions. Replay attempts fail because counters and nonces bind each response to a single outstanding challenge and communication faults are handled deterministically. These observations confirm that the protocol operates correctly on the target hardware and provides the expected guarantees under a trusted firmware model. **A complete record of the implementation, build workflow, and validation traces is available in the project repository [5].**

The project does not include the Security MicroVisor. This choice does not affect the correctness of the protocol flow, but it limits the isolation guarantees. Without $S\mu V$, the system assumes that the installed privileged firmware behaves correctly and cannot prevent an attacker with internal code execution from bypassing measurement or altering protected state. Understanding this boundary was a central goal of the work: the protocol remains correct, but its security depends on the integrity of the firmware rather than on a verified isolation layer. Adding $S\mu V$ would require an instrumented toolchain with image validation and a dedicated $S\mu V$ which is a substantially larger effort comparable to other ARM isolation designs such as PISTIS [2].

Within these assumptions, the attestation mechanism remains effective. HMAC authenticates both directions, nonces and persistent counters prevent replay, and the keyed reference measurement detects modification of the prover's flash region. The system therefore achieves its stated objectives and offers a reliable SIMPLE attestation path for deployments that trust the installed firmware but do not require the stronger isolation guarantees of $S\mu V$.

REFERENCES

- [1] M. Ammar, B. Crispo, and G. Tsudik, “SIMPLE: A Remote Attestation Approach for Resource-constrained IoT Devices,” in *Proc. 2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPs)*, 2020, pp. 247–256.
- [2] G. Grisafi, D. Cattaneo, S. Zanero, and F. Maggi, “PISTIS: Trusted Computing Architecture for Low-end Embedded Systems,” in *Proc. 2023 IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 1–15.
- [3] S. Cheshire and M. Baker, “Consistent Overhead Byte Stuffing,” *IEEE/ACM Transactions on Networking*, vol. 7, no. 2, pp. 159–172, Apr. 1999.
- [4] Intel, “TinyCrypt: A small-footprint cryptographic library,” source repository. Available: <https://github.com/intel/tinycrypt>
- [5] F. Aghi, “SIMPLE STM32 attestation implementation,” source code, build system, provisioning tools, and validation record. Available at: <https://github.com/Cobr3x/simple-implementation>.